

Sprint 4: Domain-Driven Design (DDD) — Modelagem Tática do Domínio

Projeto: Mapeamento Arquitetural de Artefatos Táticos

Sprint: Sprint 4 (Aplicação dos conceitos)

Data de Entrega: 29/05/2026

1. Estruturação Tática de Componentes no Domínio

Modelagem detalhada dos blocos de construção táticos do DDD (Entidades, Value Objects, Agregados e Repositórios) para estruturar as regras internas do ecossistema de software de forma isolada.

Componente Tático	Exemplo de Negócio do Sistema	Justificativa Arquitetural
Entidade (Entity)	ConsultaMédica (IdÚnico, Data, Status)	Possui uma identidade própria estável que persiste ao longo do tempo, independentemente de mutações em seus atributos secundários.
Objeto de Valor (Value Object)	CPF / RegistroClínico / Endereço	Não possui identidade própria. É imutável e definido inteiramente pelas propriedades conceituais que encapsula (ex: dois CPFs com números idênticos são semanticamente o mesmo objeto).
Agregado (Aggregate Root)	Prontuário (Raiz que engloba Histórico Clínico e Evoluções)	Funciona como uma barreira transacional. Apenas a Raiz do Agregado pode ser referenciada por objetos externos, blindando e mantendo a consistência interna das sub-entidades

Componente Tático	Exemplo de Negócio do Sistema	Justificativa Arquitetural
		associadas.
Repositório (Repository)	IProntuarioRepository (Interface Abstrata)	Camada abstrata que faz a ponte entre o domínio e o mecanismo de persistência de dados no banco de dados, permitindo a recuperação e persistência dos Agregados como se fossem coleções em memória.

2. Exemplo de Código Limpo para Abstração Tática (Value Object)

Demonstração em código da implementação rigorosa de imutabilidade e auto-validação em um Value Object de CPF de Paciente:

```
class CPF:
    def __init__(self, numero: str):
        if not self._validar_formato(numero):
            raise ValueError("Formato de CPF inválido para o Domínio.")
        self._numero = numero

    @property
    def numero(self) -> str:
        return self._numero

    def _validar_formato(self, numero: str) -> bool:
        # Lógica simplificada de verificação de tamanho para ilustração de regra
        de negócio
        return len(numero) == 11 and numero.isdigit()

    def __eq__(self, outro: object) -> bool:
        if not isinstance(outro, CPF):
            return False
        return self.numero == outro.numero
```